Práctica 12

Tras medir los tiempos, rellenar la tabla:

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada para los cuatro casos.

Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

n	t Bucle1 O(n*log(n))	t Bucle2 O (n^2 * log(n))	t Bucle3 O (n^2 * log(n))	t Bucle4 O(n^3)
100	0,0451	0,17	0,85	0,67
200	0,096	0,62	3,5	4,93
400	0,2	2,86	15,6	37,5
800	0,461	13,4	64	284
1600	1,02	54	266	2227
3200	2,17	253	1116	17569
6400	4,59	927	4688	FdT
12800	11,4	4151	19619	FdT
25600	22,5	18515	FdT	FdT
51200	49,2	FdT	FdT	FdT

Los datos obtenidos concuerdan, pues siguiendo el orden de complejidades (que quedarían de menor a mayor: O(n*log(n)) < O (n^2 * log(n)) < O(n^3)) a mayor es esta mayores son los tiempos obtenidos.

Lo único destacable es el caso del bucle 2 y 3, los cuales, a pesar de tener la misma complejidad, por cómo es su código y las condiciones de los bucles anidados salen tiempos diferentes.

Implementar tres nuevas clases Bucle5, Bucle6 y Bucle7, que simulen algoritmos iterativos con una complejidad O (n2 log 2 n), O (n3 log n) y O (n 4) respectivamente. Tras implementar las clases, mida sus tiempos de ejecución y rellene la tabla:

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal de los tres casos.

Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

n	t Bucle5 O (n^2 * log^2(n))	t Bucle6 O (n^3 * log(n))	t Bucle7 O(n^4)
100	0,87	7	20,1
200	4,3	64	255
400	21,6	555	3744
800	108	4774	57663
1600	532	41621	FdT
3200	2492	FdT	FdT
6400	11531	FdT	FdT

En este caso, a diferencia del anterior, no hay caso a destacar, pues son tres bucles, cada uno de mayor complejidad que el anterior, y que saca un tiempo mayor.

Tras medir los tiempos, rellenar la tabla:

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.

Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

n	t Bucle1 (t1)	t Bucle2 (t2)	t1/t2
100	0,0451	0,17	0,265294118
200	0,096	0,62	0,15483871
400	0,2	2,86	0,06993007
800	0,461	13,4	0,034402985
1600	1,02	54	0,018888889
3200	2,17	253	0,008577075
6400	4,59	927	0,004951456
12800	11,4	4151	0,002746326
25600	22,5	18515	0,001215231
51200	49,2	FdT	-

El cociente es correcto, como se aprecia, cada cociente t1/t2 con relación a N va dividiéndose (alrededor) a la mitad, debido a la forma en que crecen los respectivos bucles.

Tras medir los tiempos, rellenar la tabla:

Razone si los diferentes tiempos y sus cocientes concuerda con lo esperado.

Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

n	t Bucle3 (t3)	t Bucle2 (t2)	t3/t2
100	0,85	0,17	5
200	3,5	0,62	5,64516129
400	15,6	2,86	5,454545455
800	64	13,4	4,776119403
1600	266	54	4,925925926
3200	1116	253	4,411067194
6400	4688	927	5,057173679
12800	19619	4151	4,726331005
25600	FdT	18515	-
51200	FdT	FdT	-

En este caso, el cociente sale relativamente estable en todas las tomas, rondando entre 4,5 y 5. Saliendo mayores que 11 todos pues el bucle3, el que está como numerador, es de una complejidad mayor que bucle2, obteniendo por ende mayores tiempos.

Tras medir los tiempos, rellenar la tabla:

Razone si los diferentes tiempos y sus cocientes concuerda con lo esperado.

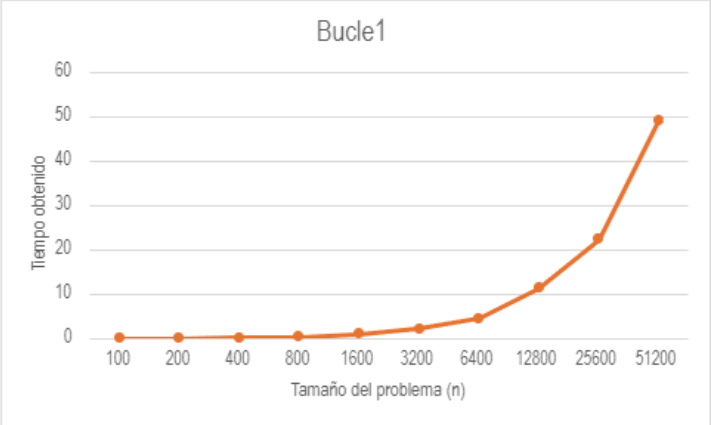
Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

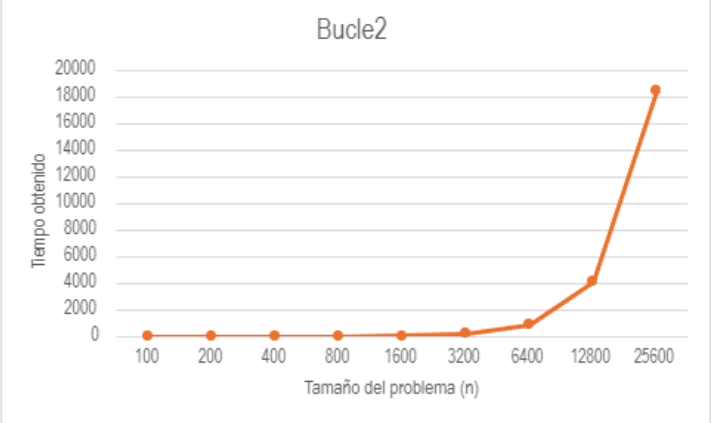
n	Bucle4(PY) t41	Bucle4(JA_SIN) t42	Bucle4(JA_CON) t43	t42/t41	t43/t42
100	4,84	0,67	0,0201	0,138429752	0,03
200	34,5	4,93	0,087	0,142898551	0,017647059
400	262	37,5	0,442	0,143129771	0,011786667
800	2201	284	2,73	0,129032258	0,009612676
1600	18762	2227	18,87	0,118697367	0,008473282
3200	FdT	17569	128	-	0,00728556
6400	FdT	FdT	854	-	-

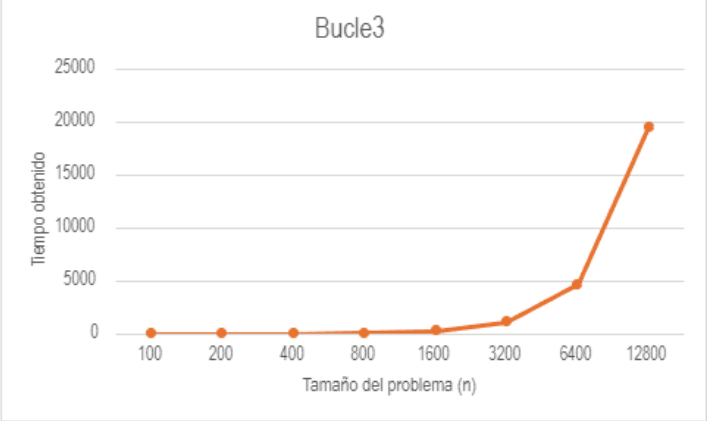
Respecto al primer cociente, el de bucle4 de Java entre su versión de Pyhton, al estar el de Java, que da menos tiempos, como numerador, los resultados son menores a 1.

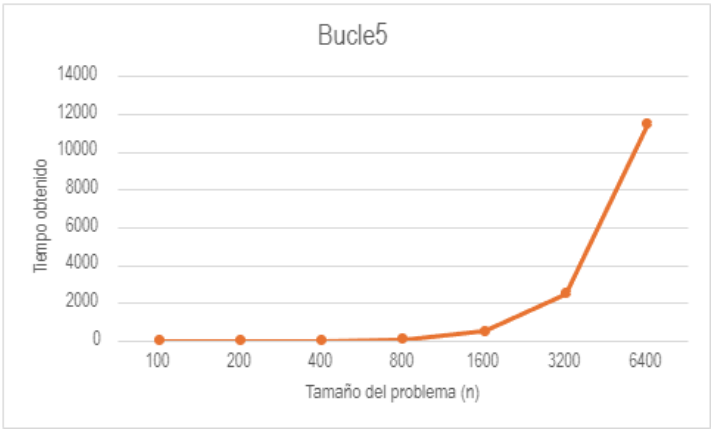
Y en cuanto al segundo cociente, al estar como numerador la versión del bucle en Java CON optimización, también salen resultados menores a 1, acercándose poco a poco a 0 según la versión optimizada aumenta su distancia respecto a la no optimizada.

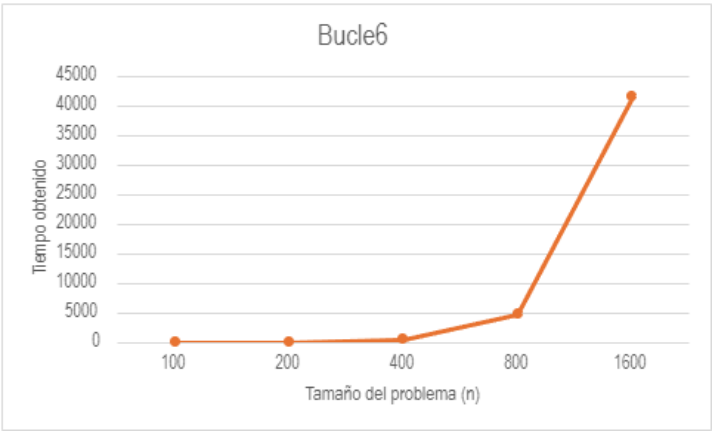
Sección de gráficas de los algoritmos empleados

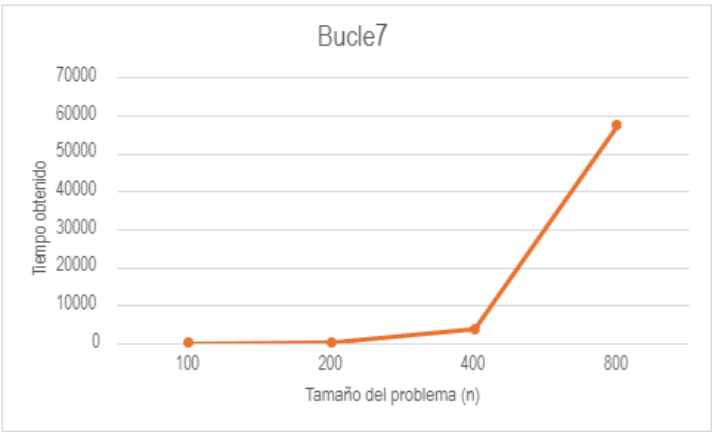







Bucle5
14000
12000
10000
8000
6000
4000
2000
0
Tiempo obtenido
100
200
400
800
1600
3200
6400
Tamaño del problema (n)


Bucle6
45000
40000
35000
30000
25000
20000
15000
10000
5000
0
Tiempo obtenido
100
200
400
800
1600
Tamaño del problema (n)


Bucle7
70000
60000
50000
40000
30000
20000
10000
0
Tiempo obtenido
100
200
400
800
Tamaño del problema (n)